

BAHRA ELECTRIC

RFP AUTOMATION PORTAL

Technical Guide — understand it, set it up, run it

Version	1.0
Date	July 2026
Audience	Developers, IT and system administrators
Classification	Confidential — internal use only

Note This guide is a plain-language companion to the full **RFP Portal Technical Manual**. It covers the same system at a level a new joiner can absorb in one sitting: what the system does, how the pieces fit together, and exactly how to get it running. It does not walk through the code file by file.

Document Control

Field	Detail
Document title	RFP Automation Portal — Technical Guide
Document number	BE-RFP-TG-002
Version	1.0
Date	July 2026
Owner	Bahra Electric — IT Department
Classification	Confidential
Companion document	RFP Portal Technical Manual (BE-RFP-TM-001) — the deep engineering reference

Who should read this

If you are...	Start at
Completely new to the system	Part 1 — it explains what the system does and how work flows through it
A developer who has just been given the code	Part 1, then Part 3 to get it running on your machine
Supporting the live system	Part 4 (the server) and Part 5 (problems and fixes)
Looking for a specific detail	The full Technical Manual — it goes deeper than this guide

A note on secrets

Passwords, client secrets and signing keys are **deliberately not printed anywhere in this document**. Where a secret is needed, the guide names the setting and tells you where to obtain the value. Treat this file as confidential even so — it describes the system's internal layout.

Contents

Document Control	2
Who should read this.....	2
A note on secrets.....	2
Contents	3
Part 1 — Understanding the System.....	5
1. What This System Does.....	5
2. The Big Picture.....	6
3. How an RFP Travels Through the System.....	7
4. The Building Blocks.....	9
Part 2 — How It Works Inside	12
5. Where Everything Is Stored	12
6. Users, Roles and Permissions.....	13
7. The Outlook Reply Card	15
8. What Runs Automatically	16
Part 3 — Running It Yourself.....	17
9. Before You Start.....	17
10. Setting Up On Your Machine.....	17
11. Configuration — What To Set And Where.....	19
12. Checking That It Works.....	20
Part 4 — The Production Server.....	22
13. How Production Is Arranged	22
14. Managing The Service	23
15. Deploying An Update.....	23
16. Rolling Back.....	24
17. Where To Look When Something Breaks	25
Part 5 — Reference.....	26
18. Common Problems And Fixes	26
19. Known Limitations	26
20. The Folder Map.....	27
21. Glossary.....	28

Tip The contents list above is a Word field. Right-click it and choose **Update Field** → **Update entire table** to fill in the page numbers.

Part 1 — Understanding the System

What the system is for, who uses it, and how work flows through it.

1. What This System Does

1.1 The problem

Bahra Electric sells cable and electrical products to large buyers — Saudi Energy, Aramco, HADEED and others. Those buyers publish their purchase requests (**RFPs — Requests for Proposal**) on the SAP Ariba supplier portal.

Done by hand, every RFP means someone has to log in to Ariba, notice the new request, download a spreadsheet of the items being asked for, work out which of our products match, email the right product specialists, collect their prices and delivery times, gather the technical documents, and finally type all of it back into the portal before the deadline. Multiply that by four buyer organisations and it is slow, easy to miss, and easy to get wrong.

1.2 What the system does instead

The RFP Automation Portal does that whole loop by itself. A software robot signs in to the supplier portal on a schedule, finds new RFPs, downloads them, works out which of our products each line item refers to, and emails an interactive card to the colleague responsible for that product. That colleague types a price and a delivery time straight into the email, attaches any technical document, and presses Submit. The system stores the answer and then goes back into the supplier portal and fills the response in for them.

A web portal sits alongside it, so the same work can also be done on screen — and so managers can see every RFP, every response and every automated run in one place.

1.3 Before and after

Step	By hand	With the system
Noticing a new RFP	Someone must remember to check the portal	Checked automatically 4 times a day
Getting the item list	Download and open the spreadsheet manually	Downloaded and read automatically
Finding our matching product	Read each line and search the material list	Matched against the Material Master automatically
Reaching the right person	Work out who owns the product, then email them	Card emailed to the owner of that product line
Collecting the answer	Chase over email and collate replies	Answer captured from the card and stored

Step	By hand	With the system
Responding on the portal	Retype everything into Ariba before the deadline	Filled in and submitted by the robot

2. The Big Picture

The system sits in the middle: people on one side, the services it depends on the other.

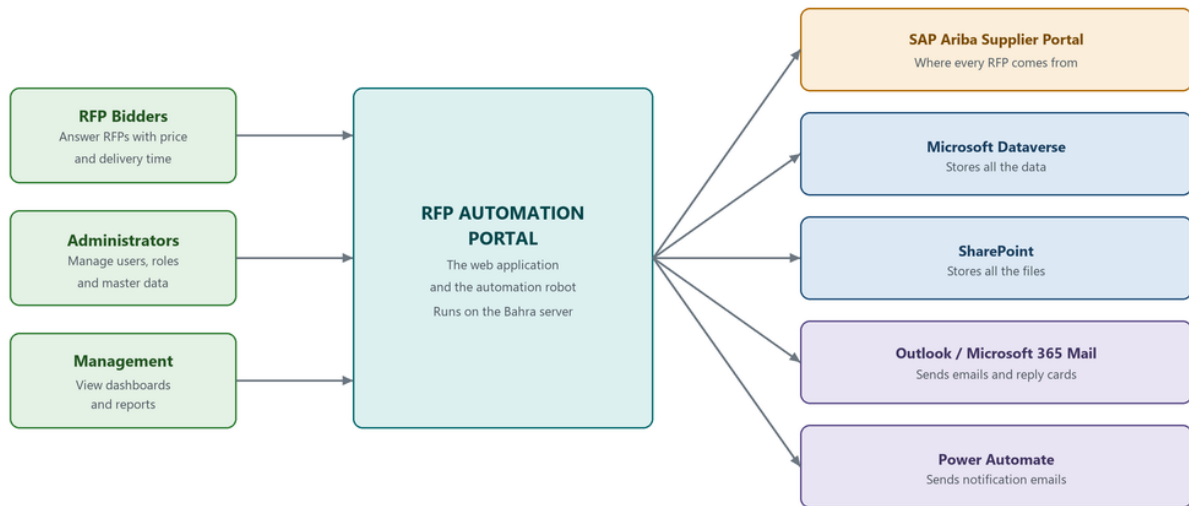


Figure 1 — The people who use the system and the services it connects to

2.1 The people

Who	What they do here
RFP bidders	Product specialists. They receive the card email and answer with a price and a delivery time, and upload technical or pricing documents.
Administrators	Create users, assign roles, maintain the material and keyword lists, and change system settings.
Management	Use the dashboard and analytics screens to see RFP volume, response rates and what the automation has been doing.

2.2 The outside services, and why each one is there

Service	What we use it for	Why this one
SAP Ariba	Reading new RFPs, downloading item lists, submitting and declining responses	It is the buyers' portal — the source of every RFP. There is no API, so the system drives it like a person would.

Service	What we use it for	Why this one
Microsoft Dataverse	Storing everything that is data: RFPs, responses, users, roles, settings, logs	It is the company's Microsoft data platform, already licensed and governed
SharePoint	Storing everything that is a file: item lists, technical data sheets, price documents, error evidence	Files belong in a document library where people can also open them normally
Microsoft 365 mail (Graph)	Sending the interactive card emails	The card has to be sent as raw email content, which only this path preserves
Power Automate	Sending plain notification emails and password-reset links	Simple, editable by the business without a code change
SAP Material Master	The reference list of our products, used for matching	It is the authoritative product list; it is exported into Dataverse

Note All the Microsoft services are reached with a **single Azure application registration**, historically named `Bahra_SharePoint`. One identity covers Dataverse, SharePoint, mail and card security. The name is misleading; the registration is not SharePoint-specific.

3. How an RFP Travels Through the System

This is the single most useful thing to understand. Everything else in the system exists to support these eight steps.

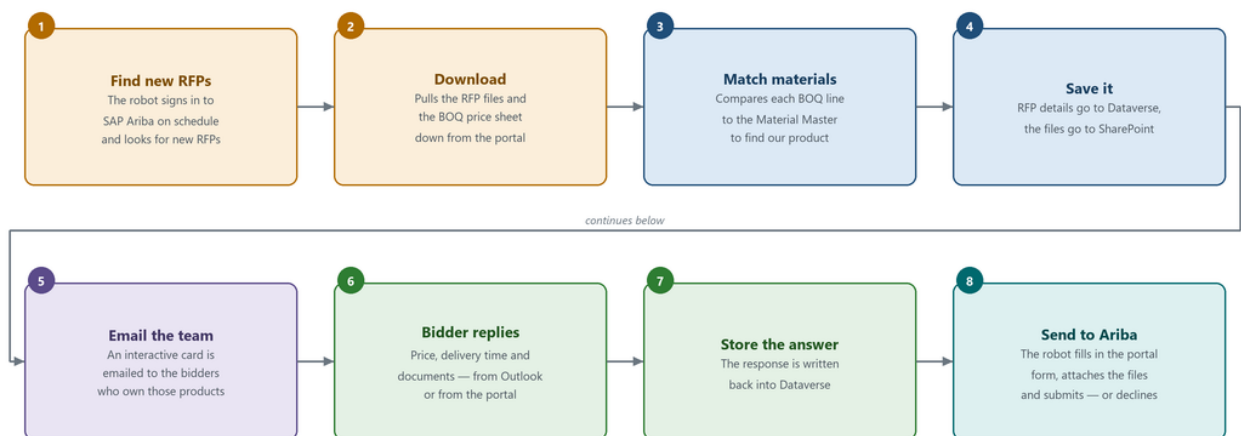


Figure 2 — The life of one RFP, from discovery to submission

3.1 The eight steps

Step 1 — Find new RFPs. Four times a day, Windows Task Scheduler tells the system to go looking. The system starts a real (but invisible) browser, signs in to SAP Ariba with the stored supplier account, and lists the RFPs for each buyer organisation.

Step 2 — Download. For each new RFP it exports the list to Excel and downloads the attachments — most importantly the **BOQ** (Bill of Quantities), the spreadsheet of line items the buyer wants priced.

Step 3 — Match materials. Each BOQ line is compared against our Material Master to work out which of our products it refers to. See section 3.3 — this matching is simpler than people expect.

Step 4 — Save it. The RFP header, its line items and the match results are written into Dataverse. The downloaded files are uploaded into a folder for that RFP in SharePoint.

Step 5 — Email the team. For every product line that matched, the system looks up who owns that product in the RFP Team list and emails them an **Adaptive Card** — an interactive form that renders inside Outlook.

Step 6 — Bidder replies. The bidder fills in price and delivery time. They can do it in the email itself, or open the portal, or use the upload link in the card to attach a technical data sheet or a price document.

Step 7 — Store the answer. The response comes back into the system and is saved against that RFP in Dataverse, along with a record of the files uploaded.

Step 8 — Send it to Ariba. When the responses are in, the robot opens the supplier portal again, walks through the multi-step response form, fills in the prices and delivery times, automatically attaches the technical files that were uploaded earlier, and submits. If we are not bidding, it declines instead.

3.2 What can happen differently

- **Nothing matched.** If no BOQ line matches any of our materials, nobody is emailed a card and a notification goes out instead. The match data is only as good as the Material Master.
- **We decline.** A bidder or an administrator can decline an RFP. The robot then clicks Decline on the portal rather than submitting.
- **Nobody replied.** Reminders can be sent from the Open RFPs screen. See section 19 — automatic reminders are currently not working.
- **Something broke.** If the robot hits an unexpected screen on Ariba it captures a screenshot and the page, bundles it up, and uploads it to SharePoint so it can be diagnosed later.

3.3 How material matching actually works

It is worth being precise about this, because the name suggests more intelligence than there is. Matching is **not** fuzzy. There is no score, no confidence value and no threshold to tune. The system does two things in order:

1. If the 9-digit material code on the BOQ line is exactly equal to one in the Material Master, that is the match.

- Otherwise it checks whether the description contains a known keyword, or the keyword contains the description — plain substring containment in either direction — and takes the first hit it finds.

Important Because there is no scoring, a short or generic keyword such as **CU** can produce a **confident but wrong** match, and nothing in the system flags it. The only way to improve matching is to improve the **Material Master and Keyword lists** — there is no algorithm setting to turn up.

4. The Building Blocks

Reading top to bottom: something makes a request, IIS receives it, the application does the work, and the outside services are used as needed.

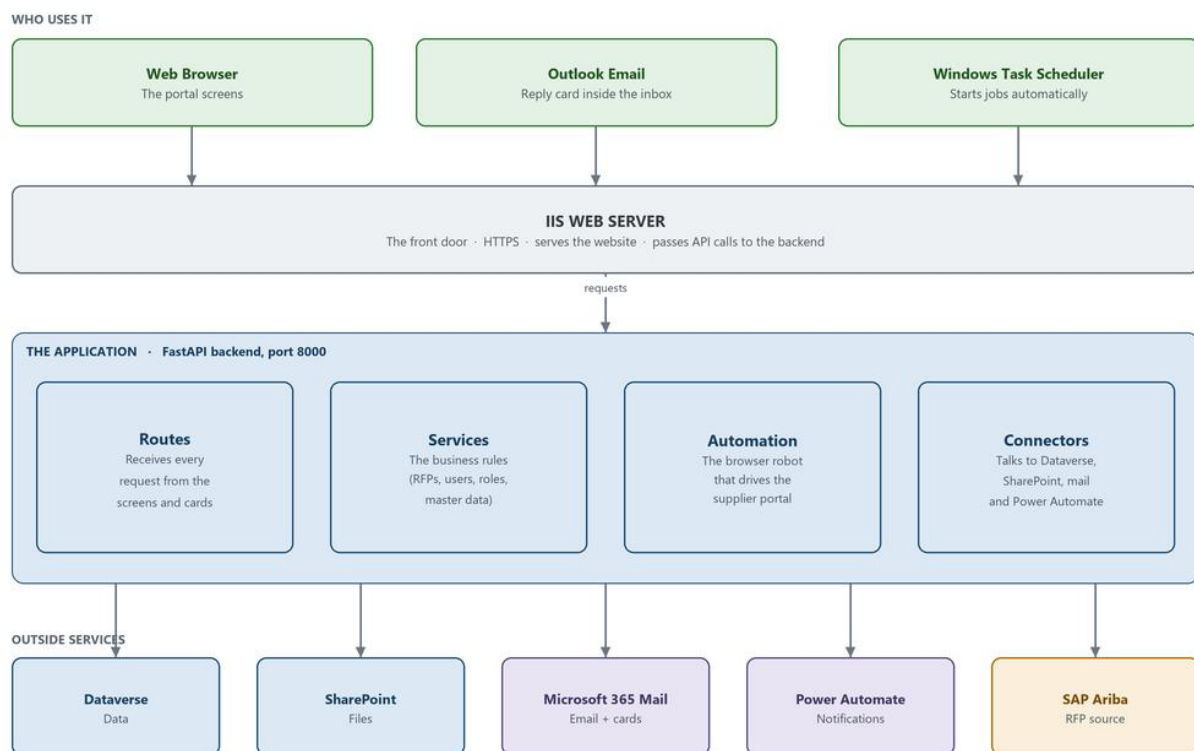


Figure 3 — How a request travels from the outside world into the application

4.1 The four parts of the backend

Part	What it is responsible for
Routes	The list of addresses the system answers on. Every screen action, every card reply and every scheduled trigger arrives here first. Routes do no real work themselves — they check permission and hand off.
Services	The business rules. What counts as a valid response, who is allowed to do what, how a user is created, how master data is imported, how the dashboard numbers are calculated.

Part	What it is responsible for
Automation	The browser robot. This is the part that drives SAP Ariba: download, submit, decline and sync. It runs on its own background thread so the website stays responsive while a job is in progress.
Connectors	The plumbing to the outside: Dataverse, SharePoint, mail, Power Automate, plus shared jobs like signing upload links and logging failures.

4.2 The website

The screens are a single-page React application. In production it is built into static files and served by IIS; during development it runs on its own dev server that forwards API calls to the backend. The screens are:

Screen	Purpose
Dashboard	Headline numbers and recent activity
RFP Insights	Every RFP with filters, detail and export
Open RFPs	RFPs still awaiting a response; send reminders, delegate a product line
Analytics / Material Insights	Charts on volume, response rates and matched materials
Logs	What the automation did on each run
Admin → Users, Roles	User accounts and what each role is allowed to do
Admin → Master Data	Materials, keywords and the RFP team routing list
Admin → System Settings, Audit, SAP Logs	Runtime settings, the audit trail and supplier-portal credential history

4.3 What it is built with, and where each piece is used

Technology	Where it is used	Why
Python 3.10	The whole backend	The automation and data libraries the project needs are Python-first
FastAPI	Every API address the system answers on	Fast to write, validates input automatically
Playwright	Driving SAP Ariba	Controls a real Chromium browser — required, because Ariba has no API
pandas / openpyxl	Reading BOQ spreadsheets	Handles both <code>.xls</code> and <code>.xlsx</code> reliably
MSAL	Signing in to every Microsoft service	Microsoft's own library for app-to-app tokens

Technology	Where it is used	Why
React + TypeScript	All the screens	Standard, well-supported, typed
Vite	Building and serving the frontend	Fast builds and a dev server with API forwarding built in
Tailwind + Radix	The look of every screen	Consistent, accessible components without hand-written CSS
TanStack Query / Table	Loading and displaying data	Caching and large tables without custom code
IIS	The production front door	Already the company's web server; terminates HTTPS
Windows Task Scheduler	Starting jobs on a schedule	Runs on the server itself; no cloud dependency

Part 2 — How It Works Inside

Where the data lives, how people are let in, and what runs without anyone pressing anything.

5. Where Everything Is Stored

There is a clean split, and it is worth memorising: **data goes to Dataverse, files go to SharePoint**. Nothing important is stored on the server's own disk — the local folders are working space only.

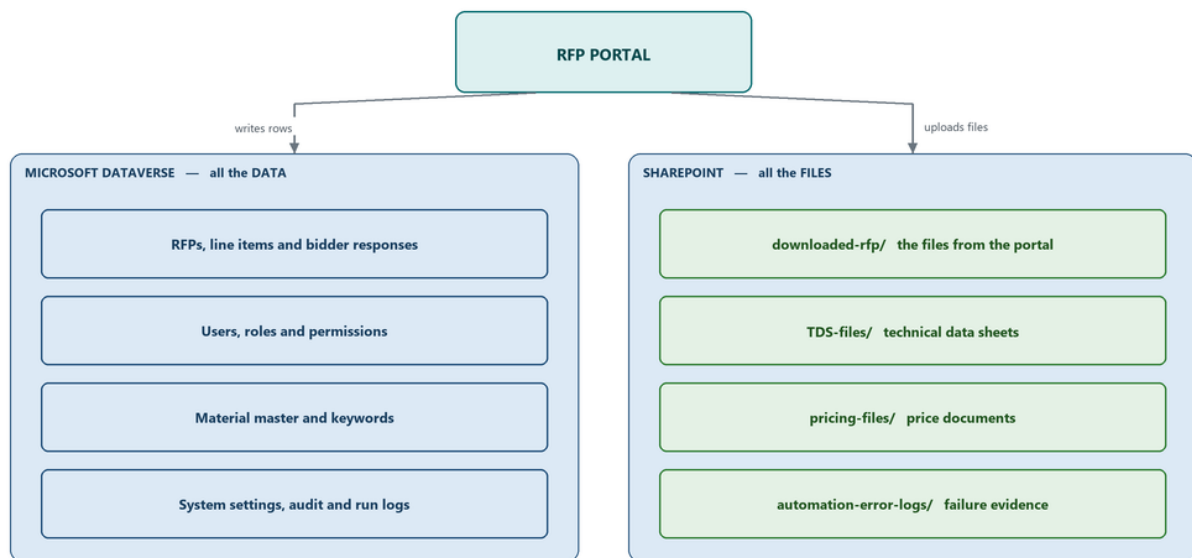


Figure 4 — The split between Dataverse and SharePoint

5.1 Dataverse — the data

Dataverse is Microsoft's managed data platform. Think of it as the database, except it is a service rather than a server we run. The tables group into five areas:

Area	What is kept there
RFPs and responses	The RFP record and its line items, the match results, each bidder's price and delivery time, delegations and reminder history
People and access	User accounts, roles, the permissions each role grants, and account status such as lockouts
Master data	The Material Master, the matching keywords, the RFP team routing list and the definitions of its columns
Settings	Runtime settings that can be changed from the portal without a code change, plus supplier-portal credentials
Logs	One row per automation run, a per-RFP activity timeline, and the security audit trail

5.2 SharePoint — the files

Every RFP gets its own folder tree in the SharePoint document library, so the files can also be opened by hand if needed.

Folder	Contents
ALLRFPs/{Company}/{RFP}/downloaded-rfp/	What was downloaded from the supplier portal, including the BOQ
ALLRFPs/{Company}/{RFP}/TDS-files/	Technical data sheets uploaded by bidders — these are attached automatically when the response is submitted
ALLRFPs/{Company}/{RFP}/pricing-files/	Price documents uploaded by bidders
ALLRFPs/{Company}/{RFP}/rfp-upload-file/	The completed spreadsheet used for submission
automation-error-logs/	Screenshots and page captures from failed automation runs

5.3 Two things about Dataverse that catch everyone out

These two quirks are behind a large share of the confusing bugs in this system, so they are worth knowing before you touch any data code.

Important Table names in the API are not predictable. Dataverse invents a plural form for each table and it is not consistent — `cr673_bahra_roles` becomes `cr673_bahra_roleeses`, while `cr673_bahra_rfp_team` becomes `cr673_bahra_rfp_teams`. Never guess it. Every table is therefore declared twice in the configuration — a real name and an API name — and the API name must be copied from what Dataverse reports, not derived.

Important Dates are already local time. Dataverse hands back timestamps that look like UTC (they end in Z) but are already Saudi local time. Converting them shifts every date by three hours. Read the numbers as they are and do not apply a timezone conversion.

6. Users, Roles and Permissions

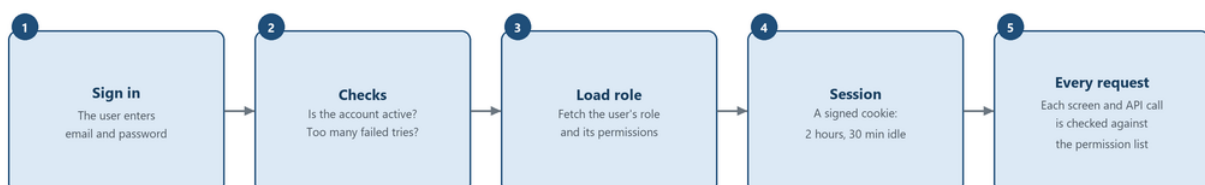


Figure 5 — From signing in to being allowed to do something

6.1 Signing in

The portal has its own user accounts — it does not use a Windows or Microsoft 365 login. When someone signs in, the system checks the password, checks the account is active and not locked, loads the permissions attached to that person's role, and issues a signed session cookie. Forgotten passwords are handled by emailing a link that is valid for 30 minutes.

6.2 Roles and permissions

Access is controlled by **role**, and a role is simply a named bundle of permissions. There are **42 individual permissions** defined, covering things like creating a user, editing master data, running an automation job or revealing a stored secret. Two roles ship with the system and cannot be deleted:

Role	Gets
Admin	All 42 permissions, granted automatically — if a new permission is added to the system, Admin gets it without any change
RFP Bidder	10 permissions — enough to see RFPs and respond to them, and nothing more

Any number of further roles can be created from the Roles screen and given whatever combination of permissions is needed.

Note Permissions are enforced on the **server**, not just hidden in the interface, so a hidden button cannot be worked around. Permission changes are cached for **5 minutes** — if a change does not appear to take effect, wait or have the user sign in again.

6.3 Password and session rules

Rule	Value
Minimum password length	8 characters
Must contain	At least one capital letter and one number
Password expires after	90 days
Failed sign-ins before logout	5
Logout lasts	30 minutes
Session lifetime	2 hours
Signed out after inactivity	30 minutes
Password-reset link valid for	30 minutes

7. The Outlook Reply Card

This is the feature people notice most, and the one with the most moving parts, so it gets its own section.

7.1 Why it exists

A bidder is a busy product specialist. Asking them to open a portal, sign in and find the right RFP adds friction and delays responses. Instead the email itself contains a small interactive form — an **Adaptive Card** — with the matched product lines, a box for the price, a box for the delivery time, and Submit and Decline buttons. It is answered without leaving Outlook.

7.2 The round trip

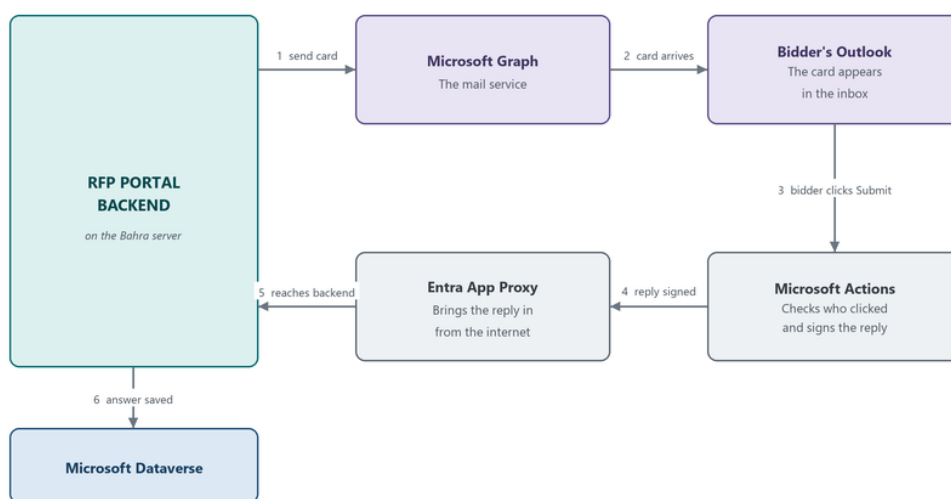


Figure 6 — How a click inside Outlook reaches the server

The interesting part is the return journey. The server sits inside the Bahra network with no inbound opening to the internet, yet Microsoft's service has to be able to deliver the reply. That is solved with **Entra Application Proxy**: a small connector on the server makes an outbound connection to Microsoft and replies come back down that same connection. No firewall port is opened and no public address is exposed.

Note The proxy must be set to **Passthrough** mode. If it is set to pre-authenticate, Microsoft's service is redirected to a sign-in page it cannot complete, and the buttons silently do nothing.

7.3 The upload link

The card also carries an **Upload** button. It opens a small page where the bidder can attach a technical data sheet or a price document — again with no login. The link is safe to send because it carries a signed token that names the specific RFP, recipient and product line, and expires after 72 hours. Files go straight into that RFP's SharePoint folder, up to 150 MB each.

Anything uploaded to the technical folder is picked up automatically when the response is submitted to Ariba, so the bidder never has to attach the same file twice.

8. What Runs Automatically

All recurring work is driven by **Windows Task Scheduler on the server**. The tasks do not do the work themselves — they call the application, which runs the job and reports back.

Job	Runs at	Allowed to take	What it does
Download	00:00, 06:00, 12:00, 18:00	90 minutes	Finds and downloads new RFPs, matches materials, emails the cards
Sync	03:00, 09:00, 15:00, 21:00	60 minutes	Re-reads the portal to update RFP status and deadlines

Important A finished job is **not** the same as a successful job. The runner reports success as long as it finished, so a run that did no useful work still looks clean. To confirm real work happened, check the automation log on the **Logs** screen, not just the scheduler history.

Part 3 — Running It Yourself

Everything needed to take the code and get a working system on your own machine.

9. Before You Start

9.1 Software you need installed

Item	Version	Note
Windows	10 / 11 or Windows Server	Required. The system uses Windows-specific date handling and a Windows-only background mode for the browser robot. It will not run on Linux or macOS as-is.
Python	3.10	Other 3.x versions are untested against the pinned libraries
Node.js	20 LTS	Comes with npm, which builds the frontend
Git	any	To pull the code

9.2 Access you need arranged before anything will work

The code on its own is not enough — the system is only useful when it can reach the company's services. Ask IT for the following before you begin:

- The **Azure application registration** details — tenant ID, client ID and client secret. One registration covers Dataverse, SharePoint and mail.
- Access to the **Dataverse environment** the application points at, so you can see the tables.
- Access to the **SharePoint site** used for RFP files.
- A **portal user account** for the RFP application itself, with the Admin role, so you can sign in once it starts.
- The **SAP Ariba supplier credentials** — but note these are read from Dataverse at runtime, not typed into a file.

Warning The application is configured to talk to a **live company environment**. Running the automation on your machine will reach the real supplier portal and can send real email. Confirm with the system owner which environment you are pointed at, and keep the email mode set to **dev** (section 11) while you are learning.

10. Setting Up On Your Machine

Six steps. Run everything from a PowerShell window at the repository root unless a step says otherwise.

Step 1 — Get the code

```
git clone <repository-url> RFP-automation
cd RFP-automation
```

Step 2 — Create the Python environment

The project expects a virtual environment in a folder called `env` at the repository root. Every command later assumes that exact location.

```
py -3.10 -m venv env
```

Step 3 — Install the backend libraries

```
env\Scripts\python.exe -m pip install -r backend\requirements.txt
```

Warning There are **two** `requirements.txt` files in the repository. Always use `backend\requirements.txt`. The older copy under `backend\Support-Files\` is incomplete — a backend installed from it fails the moment it tries to read a spreadsheet.

Step 4 — Install the browser the robot drives

Playwright needs its own copy of Chromium. This is a separate download from the Python libraries and is easy to forget.

```
env\Scripts\python.exe -m playwright install
```

Step 5 — Install the frontend libraries

```
cd frontend
npm install
cd ..
```

Step 6 — Fill in the configuration

Open `backend\config\config.py` and set the values described in section 11. This file is **not** tracked in version control, so you will not receive it with the code and a later `git pull` will never overwrite it. Get the values from IT or copy them from an existing installation.

Running it

Two processes, in two terminals. The backend **must** be started from inside the `backend` folder — that is what puts its modules on the import path and anchors its working folders. Starting it from the repository root fails, or quietly writes files in the wrong place.

```
# Terminal 1 – the backend, on port 8000
cd backend
..\env\Scripts\python.exe dashboard_main.py
```

```
# Terminal 2 – the frontend dev server, on port 3000
cd frontend
npm run dev
```

Then open **http://localhost:3000** and sign in. The dev server forwards all API calls to the backend on port 8000, so you only ever use port 3000 in the browser.

Note `dashboard_main.py` is the one and only entry point. It starts the API, the screens, the card callback and the automation, all in a single process. There is a second file called `automation_main.py` — it is legacy and nothing should be added to it.

11. Configuration — What To Set And Where

Configuration lives in two places, and knowing which is which saves a lot of confusion.

Where	What lives there	How it is changed
<code>backend\config\config.py</code>	Connection details that rarely change: the Azure app, the Dataverse address, table names, SharePoint locations, security rules	Edit the file. The application must be restarted — nothing re-reads this file while running.
System Settings screen	Values the business may need to change: email recipients, the public upload address, mail flow addresses and secrets	Change it in the portal. Takes effect after the settings cache refreshes, or immediately using the Reload Cache button.

11.1 The settings you must fill in

Setting	What it is	Where to get it
TENANT_ID	Your Microsoft tenant	Azure portal
CLIENT_ID	The application registration	Azure portal
CLIENT_SECRET	Its password	Azure portal — secret, never commit it
RESOURCE_URL	The Dataverse environment address	Power Platform admin centre
SHAREPOINT_HOSTNAME, SITE_PATH	Which SharePoint site holds the files	SharePoint

Setting	What it is	Where to get it
FLOW_URL	The notification email flow	Power Automate — copy the trigger address
FORGOT_PASSWORD_FLOW_URL	The password reset email flow	Power Automate
UPLOAD_TOKEN_SECRET	Signs the upload links	Any long random string — changing it breaks every link already sent
EMAIL_MODE	<code>dev</code> sends everything to one test address; <code>prod</code> sends to real recipients	Set to <code>dev</code> while developing

11.2 Settings worth knowing about

Setting	Default	Meaning
SESSION_TIMEOUT_SECONDS	7200	Session lifetime — 2 hours
IDLE_TIMEOUT_SECONDS	1800	Signed out after 30 minutes of inactivity
RBAC_CACHE_TTL_SECONDS	300	How long permissions are cached — why a role change can take 5 minutes
ACCOUNT_LOCKOUT_THRESHOLD	5	Failed sign-ins before lockout
ACCOUNT_LOCKOUT_DURATION_MINUTES	30	How long the lockout lasts
PASSWORD_MAX_AGE_DAYS	90	Password expiry
DASHBOARD_TTL_SECONDS	300	How long dashboard figures are cached

Important `config.py` contains real credentials and is deliberately kept out of version control. Never commit it, never paste it into a ticket, and never email it.

12. Checking That It Works

Work down this list. Each check proves the one before it.

#	Check	What a good result looks like
1	<code>curl http://localhost:8000/health</code>	Reports healthy and shows Dataverse connected — this single check proves the Azure credentials work and that a table can be read
2	Open <code>http://localhost:3000</code>	The sign-in page appears

#	Check	What a good result looks like
3	Sign in with your portal account	The dashboard loads with figures on it — proves Dataverse reads are working end to end
4	Open Admin → Master Data	Materials and keywords are listed
5	Open an RFP and click its SharePoint link	The folder opens — proves the SharePoint connection
6	Run a download job from the Automation screen	A browser window opens and drives the supplier portal, and a row appears on the Logs screen

Tip If step 1 fails, nothing else will work — fix it first. The usual causes are a wrong or expired client secret, or a [RESOURCE_URL](#) pointing at the wrong environment.

Part 4 — The Production Server

How the live system is arranged, and how to deploy, restart and roll it back.

13. How Production Is Arranged

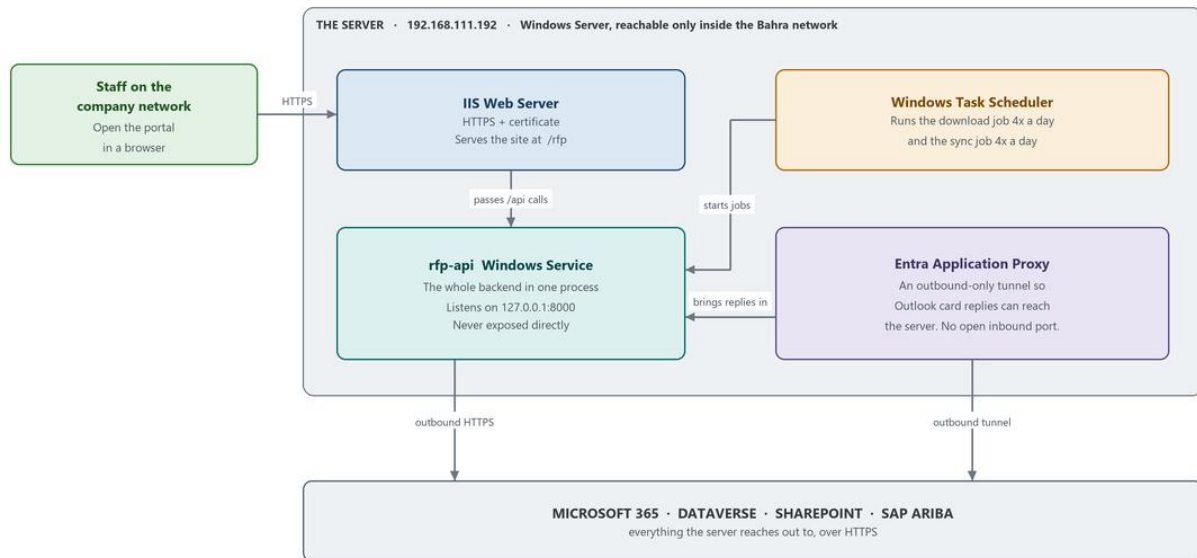


Figure 7 — The live system on the Bahra server

Item	Value
Server	192.168.111.192 — Windows Server, reachable only from inside the Bahra network
Portal address	https://be-aramco-01.bahra-cables.com/rfp
Install folder	C:\Bahra-Automation-RFP-System
Backend	Runs as a Windows service called rfp-api , listening on 127.0.0.1:8000 only
Front door	IIS — terminates HTTPS and forwards API calls to the backend
Screens	The built React files, served straight from <code>frontend\dist</code>
Scheduling	Windows Task Scheduler, in the <code>\Bahra-RFP\</code> folder
Card replies	Entra Application Proxy — an outbound-only tunnel

Important The portal is served at the **/rfp path of a shared address**, not on its own subdomain, because the same server also hosts the **COA application**. Never change COA's site, bindings or services — only the RFP site is in scope.

Note The install folder on the server is **not** the same path as a development machine. Every

command in this part assumes C:\Bahra-Automation-RFP-System.

14. Managing The Service

One service runs everything — the API, the screens, the card callback and the automation jobs. There is no separate automation service.

```
# Check it
Get-Service rfp-api

# Restart it (needed after any config.py change)
Restart-Service rfp-api

# Prove the backend itself is alive
Invoke-WebRequest http://127.0.0.1:8000/health -UseBasicParsing
```

Important Editing `config.py` on the server does nothing until the service is restarted. This is the single most common cause of "I changed it and nothing happened".

15. Deploying An Update

Run every step from an elevated PowerShell session on the server, in order. Two files are not restored by a `git pull` and are the reason step 1 exists:

- `config.py` is untracked, so a pull leaves it alone — good.
- The IIS `web.config` is **not in the repository at all**, and `npm run build` deletes it every single time. Losing it takes the portal offline.

```
# 1. Record where you are, and save the IIS config
cd C:\Bahra-Automation-RFP-System
git rev-parse --short HEAD           # write this down for rollback
Copy-Item frontend\dist\web.config $env:TEMP\web.config.bak

# 2. Stop the service and confirm it really stopped
Stop-Service rfp-api
Get-Service rfp-api                 # Status must be Stopped

# 3. Pull the new code
git pull

# 4. Update the Python libraries
.\env\Scripts\python.exe -m pip install -r backend\requirements.txt

# 5. Rebuild the screens - stop here if the build fails
```

```

cd frontend
npm install
npm run build
if ($LASTEXITCODE -ne 0) { throw 'frontend build failed - do not start' }
cd ..

# 6. Put the IIS config back into the freshly built folder
Copy-Item $env:TEMP\web.config.bak frontend\dist\web.config

# 7. Start
Start-Service rfp-api

```

15.1 Then verify

Check	Expected
Invoke-WebRequest http://127.0.0.1:8000/health	Healthy, Dataverse connected
Get-Service rfp-api	Running
Open the portal over HTTPS from a network machine	The sign-in page renders — this is what proves web.config was restored
Get-ScheduledTask -TaskPath '\Bahra-RFP\'	Both tasks present and Ready
Trigger one automation job by hand	It completes and writes a row to the automation log

Note If a release changed [config.py](#), the restart in step 7 is what makes it take effect. Neither the Application Proxy nor its connector is affected by a code deployment.

16. Rolling Back

Use the commit you recorded in step 1.

```

Stop-Service rfp-api
cd C:\Bahra-Automation-RFP-System
Copy-Item frontend\dist\web.config $env:TEMP\web.config.bak

git checkout <the-commit-you-wrote-down>
.\env\Scripts\python.exe -m pip install -r backend\requirements.txt

cd frontend ; npm install ; npm run build ; cd ..
Copy-Item $env:TEMP\web.config.bak frontend\dist\web.config

Start-Service rfp-api

```


Note Restarting during an automation run abandons that run. Nothing is corrupted — the RFP is left as it was and the next scheduled trigger picks it up — but avoid restarting while a submission is in flight.

17. Where To Look When Something Breaks

Start with the top of this list and work down. The first three answer most questions.

Look at	Where	Tells you
Logs screen	In the portal	What each automation run did, how many RFPs it handled, and the error summary
Health check	/health	Whether the backend is alive and can reach Dataverse
Service output log	Path set in the service's XML file	Request log, crashes and full error traces
Scheduler log	backend\LOGS\scheduler\	One line per trigger — proves whether a scheduled job actually fired
Robot evidence	backend\LOGS\	Screenshots and page captures from the moment a portal step failed
Failure bundles	SharePoint automation-error-logs/	The same evidence, uploaded so it can be shared
Windows Event Log	Applications → rfp-api	Service start, stop and crash codes
Task Scheduler history	\Bahra-RFP\	Whether the trigger fired and what it returned
Audit log screen	In the portal	Who signed in, who changed a user, a role or a setting

Part 5 — Reference

18. Common Problems And Fixes

Symptom	Cause and fix
A config change had no effect	Nothing re-reads <code>config.py</code> while running. <code>Restart-Service rfp-api</code> .
A scheduled run reports success but processed nothing	Playwright installs its browser per Windows account , and the service runs as a system account. Install Chromium for the account the service runs as. Re-check after every Playwright upgrade.
The portal loads but every request fails after a rebuild	<code>npm run build</code> erased the IIS <code>web.config</code> . Put it back.
Card buttons appear but clicking does nothing	The Application Proxy is set to pre-authenticate. It must be Passthrough .
A query on a new table returns nothing	The API table name is wrong. Ask Dataverse for the real one and correct the <code>_API</code> value in <code>config.py</code> — never guess it.
A permission change is not taking effect	Permissions are cached for 5 minutes. Wait, or have the user sign in again.
Emails only arrive at one test address	<code>EMAIL_MODE</code> is set to <code>dev</code> . Set it to <code>prod</code> and confirm the recipients in System Settings.
Card links or upload links stopped working	The public address changed. Update the callback address and the upload address, then restart the service.
Setup script reports "column already exists"	Dataverse reports this as a plain error rather than a conflict. The setup scripts are safe to re-run — it can be ignored.
An automation job appears stuck and will not start again	The "job is running" flag lives in memory only. Restarting <code>rfp-api</code> clears it.

19. Known Limitations

These are real, current and deliberately listed. Do not tell users a feature works when it appears in this table.

Item	What it means	Status
Automatic reminder emails do not send	Deadline reminder emails are not going out. Manual reminders from the Open RFPs screen are the only reliable route.	Open

Item	What it means	Status
The Schedule Automation screen does nothing	Saving a schedule in the portal appears to succeed but changes nothing. The real schedule is owned by Windows Task Scheduler, which that screen does not control.	Open
The card callback is published more broadly than intended	To make the card reply work, the published address covers the whole site rather than just the callback. Neighbouring addresses may therefore be reachable from outside.	Open — highest-priority security item
Material matching is not fuzzy	Exact code match, then plain substring containment. A short keyword can give a confident but wrong match. Only the master data can be improved — there is no setting to tune.	By design
The Dataverse environment carries a UAT name	The environment serving production has a UAT-style name. Confirm this is intended before any deployment.	Open
Two <code>requirements.txt</code> files exist	<code>backend\requirements.txt</code> is the correct one. The copy under <code>Support-Files\</code> is an incomplete leftover.	Open
No automated tests	There is no test suite in the repository. Changes must be verified by hand.	By design
The RBAC tables cannot be recreated from code	The four access-control tables exist in Dataverse, but the script that created them has been removed. A brand-new environment would need them built by hand.	Open

20. The Folder Map

Enough to find your way around. Folders, not files.

Folder	What is in it
<code>backend\routes\</code>	The addresses the system answers on
<code>backend\services\</code>	The business rules
<code>backend\rfp\</code>	The browser robot — download, submit, decline, reminders
<code>backend\helpers\</code>	Connections to Dataverse, SharePoint, mail and Power Automate
<code>backend\core\</code>	Shared utilities and activity logging
<code>backend\middleware\</code>	The permission checks applied to requests
<code>backend\config\</code>	Configuration — not in version control

Folder	What is in it
backend\Support-Files\	One-off setup and migration scripts. Not used by the running application.
backend\LOGS\, ALLRFPs\, automation-error-logs\	Working folders created at runtime. Nothing here is the system of record.
frontend\src\pages\	One file per screen
frontend\src\components\	Reusable interface pieces
frontend\dist\	The built site IIS serves — holds the IIS web.config
scripts\	The PowerShell scripts that register and trigger the scheduled jobs
docs\	The full documentation set

21. Glossary

Term	Meaning
Adaptive Card	The interactive form that renders inside an Outlook email, letting a bidder answer without opening the portal
Ariba	SAP's supplier network — the portal every RFP comes from
BOQ	Bill of Quantities — the spreadsheet of line items the buyer wants priced
Dataverse	Microsoft's managed data platform; this system's database
Entra ID	Microsoft's identity service, previously called Azure Active Directory
Application Proxy	An Entra service that lets an internal server receive traffic through an outbound-only tunnel
Passthrough	The Application Proxy mode that does no sign-in check — required here
Playwright	The library that drives a real browser against the supplier portal
RBAC	Role-Based Access Control — permissions bundled into roles
RFP	Request for Proposal — the buyer's invitation to quote
TDS	Technical Data Sheet — the product document attached to a response
TIR	Technical Information Requirement — a document the bidder uploads for a product line
WinSW	The wrapper that runs the backend as a Windows service

End of document